

TabUF Episode API 调用说明

给 Agent 与程序的调用合同

仓库 1587causalai/tabuf-episode-api

2026 年 8 月 15 日 v0

表怎么生成见 docs/data-generation.pdf; 本文件只讲怎么调用。

机器可读: /docs、/redoc、/openapi.json

1 给谁看 / 三十秒合同

给要正确调用本服务的 Agent 或程序。不是生成手册。

POST /v0/episodes 返回 n_{episodes} 条不同世界 (默认 8)。batch_size 默认 1: 每条自己抽 d, k , 一般不能直接 stack。要 $(8, n, d)$ 必须显式 batch_size=8。详见第 4 节。每条含 values、missing_mask、query_mask。没有额外的 y 。只拿 table 训练。

HTTP/1.1, JSON, UTF-8。无鉴权。可复现性靠 seed。掩码是 JSON boolean。values 不因 mask 挖空。单次最多 32 条。

2 基址与端点

本机基址: <http://127.0.0.1:8787>。公网隧道会变, 见附录 A。

方法	路径	作用
GET	/health	存活。{"ok": true, "version": "v0"}
GET	/v0/sources	来源目录 (canonical + 别名)
POST	/v0/episodes	抽 episode
GET	/docs /redoc /openapi.json	schema

/v0 是信封版本。换共享张量合同才升 /v1。

3 POST 请求字段

路径 POST /v0/episodes。未写的字段用默认。其它来源会静默忽略 discoscmm-only 字段。

字段	默认	范围	谁认	含义
<code>n_units</code>	1000	[8, 4096]	共享（行数）	仅 discoscm 把行当 unit；其它 $= n_{\text{samples}}$
<code>n_features</code>	null	[2, 1000]	共享	列数；null 则每个 batch 抽一次；其它来源回退 20
<code>seed</code>	0	—	共享	第 e 集用 $\text{seed} + e$
<code>n_episodes</code>	8	[1, 32]	共享	不同生成世界的条数；可大于 <code>batch_size</code>
<code>batch_size</code>	1	[1, 32]	共享	一组同形状的条数；默认每条单独一组；设 8 则可堆
<code>source</code>	discoscm	—	共享	生成器 canonical 名或别名
<code>source_name</code>	null	—	别名	别名用的子名。canonical 已自带则忽略
<code>missing_frac</code>	画像	[0, 0.95)	共享	null 则用该 source 画像
<code>query_frac</code>	画像	(0, 1)	共享	<code>label_column</code> 时无意义
<code>query_mode</code>	画像	—	共享	$\text{cells} / \text{label_column} / \text{observed_cells}$
<code>return_mechanism</code>	false	—	共享	是否附带生成说明块
<code>sigma</code>	0.3	[0, 10]	discoscm+scm	噪声标准差
<code>debug</code>	false	—	discoscm-only	额外潜分数；其它来源忽略
<code>unit_dim</code>	null	[2, 1024]	discoscm-only	k ；null 则每个 batch 抽一次
<code>type_weights</code>	70:5:10:5:5	≥ 0 , 和 > 0	discoscm-only	列类型权重（不必归一化）
<code>independent_frac</code>	0.05	[0, 1]	discoscm-only	列与 u 断开、改抽边缘的概率
<code>max_parents</code>	null	[1, 512]	discoscm-only	null: 典型上限 6 + 少量 hub；整数: 全体硬上限
<code>dag_edge_p</code>	0.3	[0, 1]	discoscm-only	遗留；仍写入 <code>response_law</code> ，新 DAG 不用 $\text{Bernoulli}(p)$

`type_weights` 五键: numeric/ordinal/binary/categorical/high_cardinality。

4 `n_episodes` 不是 `batch_size`

这两个字段最容易混。混了以后，默认调用会抽出 8 张列数不同的表，却被当成可以 stack 的一个 batch。

- `n_episodes`（默认 8）：要几个不同的生成世界。每条有自己的总体、DAG、列类型、格子、mask。第 e 条用 $\text{seed} + e$ 。
- `batch_size`（默认 1）：几个世界共用一套 (n, d, k) ，好叠成 $(\text{batch_size}, n, d)$ 。下一组才重抽 d, k （当它们在请求里是 null）。
- `n_units` 不抽，整个调用共用。

默认 `batch_size=1`：每条 episode 单独成组，于是每条自己抽 d, k 。这是故意的。只有要一步吃同形状张量时，才把 `batch_size` 写成 8。那时仍是 8 个不同世界，只是尺寸相同。

响应里 `batches` 按组切开；`episodes` 是平铺。`n_batches>1` 时，顶层不给单个 `n_features / shape`（组间不同，给了会误导）。只有一组时，顶层会重复该组的 `shape`，图省事。

4.1 三个例子

例 A（默认）。

```
{"n_units": 16, "seed": 0, "n_episodes": 8, "batch_size": 1}
```

一次返回 `n_batches=8`。某一发的形状可以是 [16, 5]、[16, 43]、[16, 12]、... (d, k 每条都重抽)。不能 stack 成 (8, 16, d)。这就是省略 `batch_size` 时的行为。

例 B (一步训练)。

```
{ "n_units": 16, "seed": 0, "n_episodes": 8, "batch_size": 8 }
```

`n_batches=1`, 例如 `shape=[16,19]`、`unit_dim=17`。8 张 `values` 可以堆成 $(8, 16, 19)$ 。内容仍不同: 8 套 U / DAG / 列类型。只锁尺寸。

例 C (两步)。

```
{ "n_units": 16, "seed": 0, "n_episodes": 16, "batch_size": 8 }
```

`n_batches=2`。某一发: 组 0 形状 $[16, 5]$ 、 $k = 32$; 组 1 形状 $[16, 43]$ 、 $k = 26$ 。组内可叠, 组间换形状。`batches[0].episodes` 是前 8 条, `episodes` 仍是平铺的 16 条。

写死 `"n_features": 20` 则所有组的 d 都是 20, 不再抽列数。`unit_dim` 同理。

5 响应形状

成功时 HTTP 200。 `response_model_exclude_none=true`: 值为 `null` 的可选键不出现。

```
{
  "n_episodes": 8,
  "batch_size": 1,
  "n_batches": 8,
  "n_units": 16,
  "batches": [{ "batch_index": 0, "shape": [16, 8], "episodes": ["..."] }, "..."],
  "episodes": [{
    "seed": 0,
    "table": {
      "n_units": 16, "n_rows": 16, "n_features": 8, "unit_dim": 4,
      "source": "discoscm",
      "values": [...], ...],
      "missing_mask": [[false, ...], ...],
      "query_mask": [[true, ...], ...],
      "column_types": ["numeric", ...],
      "n_classes": [null, ...],
      "query_mode": "any_cell",
      "shapes": {
        "values": [16, 8],
        "missing_mask": [16, 8],
        "query_mask": [16, 8],
        "n_missing": 6, "n_query": 19, "n_query_and_missing": 1
      }
    }
  }]
}
```

`column_types`: `numeric` / `ordinal` / `binary` / `categorical` / `high_cardinality`。没有 `y_query` / `y_input` / `context_mask`。`return_mechanism=true` 时可能多出第 7 节的键。

6 来源目录

以 `canonical` 名为准。`GET /v0/sources` 返回同样的画像。未知名 $\rightarrow$ 422, detail 列出 `canonical`。

canonical	别名	行含义	query	missing	HTTP
discoscm	—	unit	any_cell 0.15	0.05	200
scm	—	iid_sample	label_cell 0.15	0.05	200
sklearn_make_classification	synth*	iid_sample	label_cell 0.15	0.05	200
sklearn_make_regression	synth*	iid_sample	label_cell 0.15	0.05	200
sklearn_friedman1	synth*	iid_sample	label_cell 0.15	0.05	200
sklearn_low_rank	synth*	iid_sample	any_cell 0.15	0.05	200
sklearn_iris	real†	entity_row	label_cell 0.15	0.05	200
sklearn_wine	real†	entity_row	label_cell 0.15	0.05	200
sklearn_breast_cancer	real†	entity_row	label_cell 0.15	0.05	200
sklearn_diabetes	real†	entity_row	label_cell 0.15	0.05	200
openml	—	entity_row	label_cell 0.15	0.05	501
recsys	—	user	any_cell 0.15	0.05	501

* 别名 sklearn_synthetic。† 别名 sklearn_real。

别名用法:

- sklearn_synthetic: source_name 为 make_classification, make_regression, make_friedman1, make_low_rank_matrix。省略则随机抽一个, 并套用该 canonical 画像。
- sklearn_real: source_name 取 iris / wine / breast_cancer / diabetes; 省略则随机。

any_cell: 按 query_frac (默认 0.15) 在全表抽 query 格子。label_cell: 只在目标列 (默认最后一列, 可用 query_column) 上按同样比例抽行; 其它列不当 query。sklearn_low_rank 无标签列, 走 any_cell。旧名 cells/observed_cells、label_column 仍接受。

7 return__mechanism: 哪些键出现

return_mechanism=true (discoscm 上 debug=true 同样附带) 时, 按 source 出现不同键。不要解释这些块; 不要喂进网络。

source	会出现	不会出现
discoscm	population(mixture+assignment+representation)+ response_law	mechanism
scm	mechanism: framework=ANM-SCM, edges, node_fn, target_col	population、response_law
sklearn_*	mechanism: maker 或 dataset	population、response_law
openml / recsys	—	整条请求 501, 无 episode

population.mixture: n_components= M , M 在 $1 \dots \lfloor \sqrt{n} \rfloor$ 上 $P(M = m) \propto 1/m$; 每个分量 family 为 gaussian 或 cauchy。assignment[i] 是 unit i 的分量下标。含义见生成手册附录 A.2。

8 可复制 curl

把基址换成当前服务地址。默认空 body = discoscm。

```
curl -s http://127.0.0.1:8787/health

curl -s http://127.0.0.1:8787/v0/sources
```

```
curl -s -X POST http://127.0.0.1:8787/v0/episodes \
-H 'Content-Type: application/json' -d '{}'
```

```
curl -s -X POST http://127.0.0.1:8787/v0/episodes \
-H 'Content-Type: application/json' \
-d '{"source":"scm","n_units":32,"n_features":8,"seed":1,"return_mechanism":true}'
```

```
curl -s -X POST http://127.0.0.1:8787/v0/episodes \
-H 'Content-Type: application/json' \
-d '{"source":"sklearn_iris","n_units":30,"n_features":5,"seed":0}'
```

9 错误码

码	何时
200	成功。body 为上一节的 JSON。
422	字段越界 (n_units/n_features/unit_dim/sigma 等 le=/ge=) ; 未知 source (detail 列出 canonical 名) ; 非法 source_name; type_weights 之和 ≤ 0 。
501	openml 或 recsys: 已占位, 缓存未接。
404	路径不存在。
405	方法不对 (例如 GET /v0/episodes)。

10 不要做

- 不要把 population / response_law / mechanism 喂进网络。只训 table。
- 不要期望 y_query / y_input / context_mask。真值在 values 的 query 格。
- 不要把 n_units 一律当成 unit。只有 source=discoscm 把行解释成 unit; 其它来源它是行数 / n_{samples} 。
- 不要把 n_episodes 当成 batch_size 的别名。默认 batch_size=1, 8 条表的列数可以全不同; 要叠张量必须显式写 batch_size。
- 不要把 batch_size=8 理解成「同一张表复制 8 份」。那是 8 个世界、同一套 (n, d, k) 。

A 公网隧道

会变。当前: <https://spotlight-predicted-heaven-clips.trycloudflare.com>。优先用本机 <http://127.0.0.1:8787>。